

Project 13 (Bonus) — CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass

Title: CodeGuard Multi Agent PR Review Security Style And Test Coverage In One Pass

Type: Multi-agent (4 agents: 1 Supervisor + 3 reviewers) · **Stack:** Python, LangChain, LangGraph, RAG, FastAPI, Docker · **Level:** Advanced (a test of what you've learned) **Tagline:** A supervisor-led team of three specialist reviewers — Security, Style, and Test-Coverage — that turns a raw code diff into one compiled review comment.

Overview

CodeGuard is a Supervisor-led team of three specialist reviewer agents — Security, Style, and Test-Coverage — that turns a raw code diff into one compiled, sorted review comment. It's the same multi-agent Supervisor pattern used for content research and publishing systems, pointed at a different problem: reviewing pull requests instead of writing content. The Style reviewer is grounded in a real, searched style-guide document via RAG instead of guessing from general model knowledge, and the Supervisor routes each diff only to the reviewers that actually apply before merging their findings into one clear comment. This is the kind of first-pass check that runs before a human reviewer opens a pull request, on any team where secrets, style, and test coverage are worth catching automatically.

Features

- Three independent specialist reviewer agents — Security, Style, Test-Coverage — each tested standalone before joining the team
- Style reviewer grounded in a real, searched style-guide document through RAG, not general model knowledge
- Supervisor routes each diff only to the reviewers that actually apply, instead of running every reviewer on every diff
- Compiles all three reviewers' findings into one de-duplicated, sorted review comment instead of stitching reports together
- Limits every reviewer's input to a diff's actually-changed lines, avoiding flags on pre-existing, unrelated code
- Optional production wrap (FastAPI, database, Docker, test-gated release) reusing a standard deployment pattern

Tech Stack

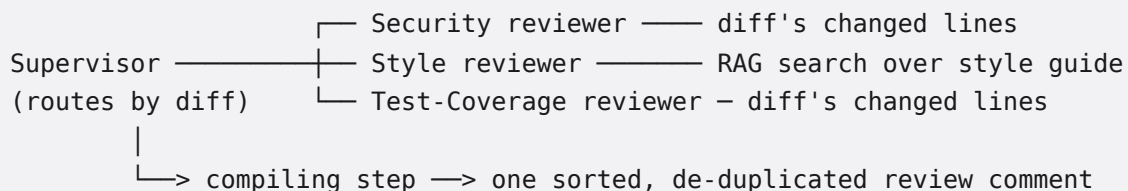
- Python
- LangChain
- LangGraph
- RAG
- FastAPI
- Docker

Prerequisites

- Comfortable with LangChain/LangGraph multi-agent patterns, including Supervisor routing to specialist agents
- Understand RAG — retrieving from a real document instead of relying on a model's general knowledge
- Know how to build a chain with LangChain (prompts and output parsing)
- Familiar with FastAPI and Docker if you plan to do the optional production wrap (Step 4)

Architecture

A Supervisor receives a code diff and routes it to whichever of the three specialist reviewers actually apply — for example, skipping the Security reviewer on a docs-only change. The Security and Test-Coverage reviewers work directly from the diff's changed lines; the Style reviewer additionally searches a real style-guide document through RAG before answering, so its findings are grounded in the team's actual document instead of the model's general training knowledge. Each reviewer returns its findings independently, and the Supervisor's compiling step merges them into one review — removing overlapping findings, sorting by severity, and writing a single clear comment instead of three reports stuck together. An optional Step 4 wraps the finished team in the same FastAPI/database/Docker production layer used elsewhere.



Setup (do this once, before Step 1)

```
cd project_13_codeguard
python -m venv .venv
source .venv/bin/activate
pip install openai langchain langchain-openai langgraph chromadb fastapi "uvicorn"
pip freeze > requirements.txt
```

Same `.env` setup as every other project (`OPENAI_API_KEY` , `LOG_LEVEL`). You'll also need a small style-guide document (write 1-2 pages yourself, or use a real one from a project you know) to build the RAG knowledge base from.

Don't have a private repo with real pull requests handy? Any public GitHub repo works fine for practice — clone one with recent PRs, or use one of your own.

Troubleshooting

Problem	Fix
The Security reviewer flags something in code that didn't actually change (an old issue, not caused by this diff)	Limit every reviewer's input to <i>only</i> the diff's added/changed lines, not the whole file
The Style reviewer disagrees with your actual style guide, because it's answering from general training knowledge instead of your specific guide	This is exactly RAG's job — search your actual style-guide document, don't let the reviewer freelance from what it already "knows"
The Supervisor runs all 3 reviewers on every diff, no matter what, wasting money	Route based on which files actually changed — a <code>.md</code> - only diff doesn't need a security check
The compiled final review is just the 3 reviewers' output stuck together, hard to read	The Supervisor's compiling step is a real task on its own — remove overlapping findings, sort by how serious they are, and write one clear comment, not a copy-paste of three separate reports

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

How To Build This — Step by Step

On purpose, less detailed than the earlier projects — you've built four multi-agent systems by now. This document trusts you to break each step into your own smaller steps, the way earlier projects did it for you.

Step 1 — One Reviewer, Working as a Plain Chain on a Made-Up Diff

*Project: CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass —
Step 1 of 4: One Reviewer, Working as a Plain Chain on a Made-Up Diff*

What this step does: proves you can figure out this project's own version of "Step 1" from a one-line description now, without me spelling out smaller steps — the way applying core LangChain/LangGraph concepts yourself would suggest. **Why this step matters:** the actual skill this whole bonus project is testing is starting from a one-line description and building your own first step — proving that on the smallest possible piece here, before anything else depends on it, is what makes the rest of the project trustworthy. **What's new:** nothing exists yet — this is the start. **What stays the same going forward:** whatever chain you build here becomes the reused core of your Security (or whichever you pick first) reviewer agent. **When you'll hit this for real:** any time you're handed a new problem domain and have to figure out your own first step — which is most of a real job, once you're past following tutorials. **Helpful background (if needed):** LangChain fundamentals — chains and prompts; tool-calling, if your reviewer needs a real tool (like a text-search secret scanner) instead of just something you prompt for.

Pick one reviewer (Security is a good start — clear, checkable rules) and get it answering correctly on 2-3 made-up example diffs before anything else exists. Include at least one diff that's clean on purpose, not just diffs with a planted problem — a reviewer you've only ever tested against bad input will happily flag something in code that's actually fine.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Step 2 — That Reviewer, Turned Into an Agent; Add the Style Reviewer With RAG

*Project: CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass —
Step 2 of 4: That Reviewer, Turned Into an Agent; Add the Style Reviewer With RAG*

What this step does: brings in review grounded by search (your style guide), and proves two separately-built reviewers can exist side by side without getting in each other's way. **Why this step matters:** an ungrounded Style reviewer answering from general training knowledge instead of your real document would look correct right up until it confidently cites a rule that isn't actually yours — this step is what makes that failure mode checkable instead of invisible. **When you'll hit this for real:** any review or QA tool checking against a written standard (style guide, compliance rules, brand voice) — the model needs to check against your real document, not its own training knowledge. **Helpful background (if needed):** RAG — retrieving from a real document instead of relying on the model's general knowledge.

Build the Style reviewer against a real, searched style guide (not the model's general knowledge — this is the whole point, see the problems/fixes table above). Test both reviewers separately, against the same set of example diffs. Write at least one style rule into your guide that's specific or unusual enough a model wouldn't invent it from general training knowledge on its own — a generic rule any model would get right regardless proves nothing about whether retrieval is actually working.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Step 3 — Add the Test-Coverage Reviewer + Supervisor Routing (final: 4 agents)

Project: CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass — Step 3 of 4: Add the Test-Coverage Reviewer + Supervisor Routing

What this step does: finishes the team and adds the coordination layer — routing, compiling results, and the actual multi-agent value. **Why this step matters:** 3 separately-correct reviewers stuck together is not the same thing as one useful review — this step is where the actual value of a multi-agent system over 3 standalone tools either shows up or doesn't, which is the whole thing this project exists to prove. **When you'll hit this for real:** any "combine several specialist opinions into one clear recommendation" system — code review, content moderation, multi-criteria approval workflows all have this exact shape. **Helpful background (if needed):** multi-agent coordination — routing work to specialists and compiling their results.

Build the Test-Coverage reviewer, then the Supervisor: routing based on the diff (which reviewers actually apply), a compiling step that merges findings into one clear review, and — reusing the standard revision-loop idea if you want to go further — an optional pass where the Supervisor can ask a reviewer to double-check an unclear finding before finishing. If you build that optional pass, give it a hard round limit the same way any Generator ↔ Reviewer loop needs one — an "ask for clarification" loop with no limit is the exact same infinite-loop risk in a new shape.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Step 4 — Production Wrap (optional, reuses the standard deployment steps completely)

Project: CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass — Step 4 of 4: Production Wrap

What this step does: wraps the finished 4-agent system in a production layer — FastAPI, a database, Docker, and a test-gated release process — proving that layer generalizes too, not just the agent design. **Why this step matters:** it's the difference between a project you can only describe and one you can hand someone a real link to — and doing it a second time, by reusing that production layer almost unchanged, is what actually makes those production patterns stick as reusable skills instead of one-off code you happened to write once. **When you'll hit this for real:** any project you want a real, runnable link for instead of just source code — this is what makes CodeGuard something you can point an interviewer at, not just describe.

If you want this as a real, runnable API: wrap it in FastAPI, add a test suite (a set of example diffs with known-correct findings), and containerize it — literally repeat the same production-wrap steps, on this codebase instead of the original one. No new ideas here — just repetition, which is the fastest way to make those production patterns stick permanently. Include at least one eval task that specifically checks Step 3's routing (e.g. a docs-only diff correctly skipping the Security reviewer) — a suite that only checks each reviewer's accuracy could still pass even if routing were completely broken.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Final Deliverable: CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass — a Supervisor-led team of 3 specialist reviewers that turns a code diff into one compiled, sorted review comment, grounded in your actual style guide through RAG.

Checklist Before You Call This Done

- ☐ 3 independent reviewer agents (Security, Style, Test-Coverage), each tested on its own before joining the team
- ☐ The Style reviewer is grounded in a real, searched style-guide document, not general model knowledge
- ☐ The Supervisor routes based on the diff — not every reviewer runs on every diff
- ☐ The final output is one compiled, cleaned-up, sorted review — not three reports stuck together
- ☐ You designed this project's own step breakdown, design document, and file layout, without them being handed to you in full detail

Status

Not started. Track your own progress however works for you.